

Edge cases (where the ¾-word rule breaks) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Text	Word-count guess	Actual (tokenizer tool)
<code>print('hello world')</code>	~3 tokens	usually 5-7 tokens
"میں آج بازار جا رہا ہوں" (Urdu, "I am going to the market today")	~7 tokens	often 15-20+ tokens

Code costs more than it looks like because punctuation (`(` , `'` , `)`) usually tokenizes as its own piece, not "for free" the way a space between English words is. Non-English text costs more because the tokenizer's vocabulary was built mostly from English training data — a script with less representation there gets broken into smaller, less efficient pieces to represent the same meaning.

The one-line rule to actually use: if the content isn't plain English prose (code, another language, dense structured data like JSON), stop estimating and run it through the real tokenizer tool or `tiktoken`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Code: `print('hello world')`

naive word-count estimate (2 "words" + punctuation) : ~3 tokens
 actual : 5-7 tokens

Likely token breakdown: `print` , `(` , `'` , `hello` , `world` , `'` , `)` — roughly one token per syntactic element, because code's punctuation carries meaning the tokenizer's training data treated as worth its own token, unlike a plain-English comma that usually merges into a neighboring word's token.

Non-English: "میں آج بازار جا رہا ہوں"

English-equivalent estimate ("I am going to the market today", ~7 tokens) : ~7 to
 actual : often

The mechanism: byte-pair-encoding-style tokenizers build their vocabulary from the most frequent character sequences in a training corpus. A corpus that's mostly English text produces a vocabulary rich in efficient whole-word and common-subword English tokens. A script underrepresented in that training data has few or no such efficient chunks reserved for it, so the tokenizer falls back to much smaller sub-word or even byte-level pieces to represent the same text.

Difference from Basic: Basic gives you the numbers and the gap. Intermediate explains the mechanism (byte-pair encoding trained on a mostly-English corpus) that produces that gap for both code and non-English text — the same underlying cause, showing up as two different-looking symptoms.

Why this matters practically, beyond trivia: a system prompt full of JSON-schema examples or embedded code tokenizes far worse than its word count suggests, inflating cost on every single call it's part of. A multilingual product priced the same per user regardless of language will quietly cost more per conversation for non-English users — worth knowing before you build a pricing model or a context-budget assumption around English-only testing.

Which one should you actually write? For exploring a new content type once, use the browser tokenizer tool — it's fast and needs no code. For anything shipped in a real feature, call `tiktoken`

directly wherever a prompt, template, or user-facing string is built, instead of relying on the $\frac{3}{4}$ -word rule for content that isn't plain English prose.